APP de salas de chat

Funcionalidad

Como proyecto hemos creado una aplicación de salas de chat. En esta aplicación, cada cliente selecciona su nombre de usuario y, a partir de ahí, le aparecen el resto de clientes conectados en ese momento con los que puedes chatear, así como las salas de chat ya en funcionamiento. De esta manera, el cliente puede decidir abrir una nuev+a sala para hablar con uno, dos o el número que quiera de usuarios de la aplicación, o bien el cliente puede decidir unirse a una sala ya existente para hablar con los usuarios dentro de esa sala. De forma global las funcionalidades que presenta nuestra aplicación pueden resumirse con el mensaje de información de la app que le aparece a cada usuario cuando se conecta:

```
Le recordamos las funcionalidades de las que disponemos:
- type '/join' y el nombre del chat para unirse a ese chat
- type '/add', el nombre del chat y después el nombre de usuarios separados por espacios para agregarlos al chat
- type '/create' y el nombre del chat para crear un nuevo chat
- type '/leave' para salir del chat
- type '/info' para que le vuelva a salir este mensaje
- type '/conexion' para volver a ver los chats abiertos y personas conectadas
- type '/exit' para desconectarse de la app
```

Extremos necesarios

Para conseguir las funcionalidades expuestas se ha hecho uso de un extremo servidor que manejará varios extremos de clientes de forma simultánea, todo ello siguiendo el protocolo TCP.

Parámetros de entrada

Se ha diseñado el servidor TCP para que al ejecutarse necesite cómo parámetro de entrada el número de puerto en el que se quiere empezar la escucha a posibles clientes. Asimismo, en cuanto al cliente, a parte de necesitar el puerto, también requiere como primer parámetro de entrada la ip del servidor con el cual se quiere establecer la conexión.

Resultados de salida

Una vez establecida la conexión entre cliente y servidor en el número de puerto especificado, el cliente va a obtener como resultados de salida distinta información. El servidor le enviará dos tipos de mensajes según los eventos ocurridos. Por un lado, al cliente le llegarán mensajes de la aplicación en sí como, por ejemplo, la información de qué salas de chat hay abiertas o cuáles son los comandos útiles de la app. Además, cuando el cliente esté unido a una sala, el servidor hará de pasarela de comunicación para hacerle llegar al cliente los mensajes enviados por el resto de miembros de la sala indicando en cada momento qué usuario ha mandado cada mensaje.

Número de sockets utilizados

En primer lugar, por cada cliente que se une se crea un socket cliente con los dos argumentos introducidos: ip y número del puerto del servidor.
Por otro lado, el servidor maneja estos múltiples sockets de clientes abriendo un único socket servidor a partir del argumento introducido del número de puerto. Este socket de servidor está escuchando mediante un bucle infinito las posibles conexiones y va aceptando cada uno de los sockets cliente abiertos. Tras aceptarlo, los deriva a la clase MultiServerThread para que los trate de forma individualizada, pero simultánea, al comunicarse con el cliente leyendo y escribiendo sobre el socket de conexión.
En cuanto al cierre del socket, se ha hecho uso de una finalización a medias de la conexión, donde solo se cierra el socket de conexión por parte del extremo de servidor. Cuando un usuario escribe "/exit", el MultiServerThread se encarga de realizar el cierre solo del socket de ese cliente, permitiendo mantener la conexión del resto de usuarios.

Comunicación por sockets

Para la comunicación entre sockets, en el extremo del cliente se ha optado por leer el texto asociado a la entrada por teclado (`InputStreamReader(System.in)`) y convertirlo a un stream de bytes salientes (`DataOutputStream(clientSocket1.getOutputStream())`) a la hora de enviarlo al extremo de servidor. Respecto a la recepción de la respuesta por parte del servidor, el cliente crea un stream de texto asociado a cada respuesta entrante del socket (`InputStreamReader(clientSocket1.getInputStream()))`.

Por otro lado, respecto al extremo del servidor, para cada hilo de ejecución del MultiServerThread se leen los datos de texto entrantes por parte del cliente (`DataOutputStream(clientSocket1.getOutputStream())`) y las respuestas pertinentes por parte del servidor se envían vía bytes salientes que se imprimen en formato texto (`PrintWriter(socket.getOutputStream(), true)`).

Concurrencia de la aplicación

Como se ha ido presentando previamente, para el tratamiento de varios usuarios de forma simultánea, se ha optado por utilizar el método de Threads. Para ello, se ha creado la clase MultiServerThread que maneja cada cliente dentro de su propio hilo de ejecución individualizado. En consecuencia, cuando un cliente se conecta, el servidor crea un hilo para tratar ese socket y, mientras el cliente no escriba "/exit" para abandonar la aplicación, se tratará su conexión según lo especificado dentro de la función *run().*